



LIONSDEV

Curso de Programação

MÓDULO 1

Introdução à Programação e Ambiente de Desenvolvimento

Professor Nicolas Cardoso Motta

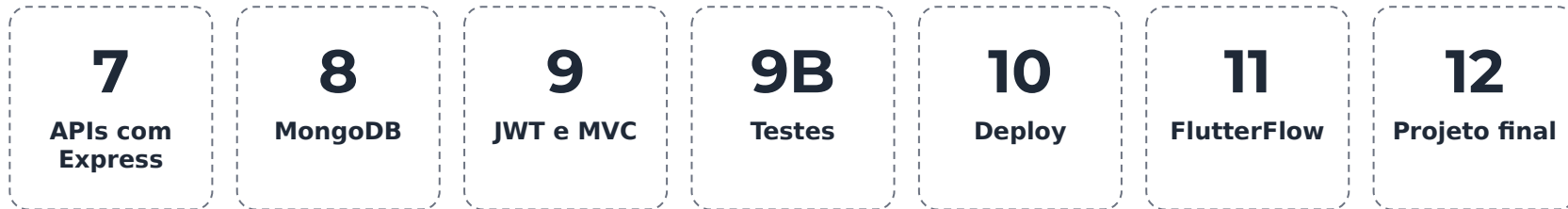


Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO



Você está aqui: **Módulo 1**, Introdução à Programação e Ambiente de Desenvolvimento



Objetivo

- **Conhecer a turma, a metodologia e como o curso funciona.**
- **Entender o que é programação e o que compõe um ambiente de desenvolvimento.**
- **Instalar e conhecer o VS Code, o Node.js e o npm.**
- **Usar o terminal do Linux para navegar e manipular pastas e arquivos.**
- **Personalizar o VS Code com tema, atalhos e extensões.**
- **Criar um projeto Node.js e instalar pacotes com o npm.**



Apresentações

Sejam bem-vindos à LionsDev!

Prazer, eu sou o Nicolas!

Vou ser o professor de vocês neste curso.



Formação

Graduando em Ciência da Computação na UTFPR.



Projetos

Focado em construir projetos dentro da universidade e pessoais. Estamos construindo a primeira empresa júnior de Ciência da Computação e ADS de Ponta Grossa.



Onde me encontrar

LinkedIn: [linkedin.com/in/nicolassmotta](https://www.linkedin.com/in/nicolassmotta)

GitHub: github.com/nicolassmotta

E você?



Como é seu nome?



Quais são seus hobbies? O que você gosta de fazer?



O que te motivou a se inscrever no curso?



Você já teve alguma experiência com programação?



Do Zero ao Herói do Código

A trilha JavaScript da LionsDev

Metodologia: Problem Based Learning



Problemas reais

Desafios que refletem o mundo real da programação web e mobile.



Resolução

Resolver os problemas aplicando os conceitos aprendidos.



Facilitação

Espaço para explorar soluções criativas.



Prática e reflexão

Atividades que levam a teoria para a prática.



Avaliação

Investigar problemas, trabalhar em equipe e apresentar soluções.



Feedback contínuo

Autoavaliação e avaliação por pares.

Ementa



Unidade I

Fundamentos da programação em JavaScript

Ambiente de desenvolvimento, fundamentos e debugging com Node.js, atividades práticas, Git e GitHub.



Unidade II

Programação intermediária

Sistema de cadastro e busca, API, MongoDB, deploy no Render e FlutterFlow.



Unidade III

**Projeto de conclusão
Solução de problemas reais e desenvolvimento de soft skills.**

Aulas intercaladas a cada 15 dias durante todo o curso.

Avaliação



Avaliações

Haverá avaliações nas Unidades I, II e III.



Média 7

A média para aprovação é 7 pontos de 10.



Presença mínima de 75%

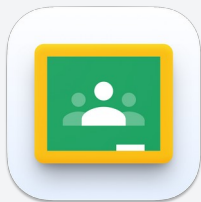
A frequência é cobrada em todas as unidades.



Soft skills

Participação e presença também são avaliadas.

Onde fica o material do curso?



Google Sala de Aula

Código da turma

jc4jza25



Material de apoio no GitHub

Conteúdo, listas e exercícios
resolvidos

`github.com/nicolassmotta/curso-lions-dev`



Introdução à Programação

O que é programação?



Instruções para o computador

De forma simplificada, é o processo de criar instruções para que um computador execute tarefas específicas.



Linguagens de programação

Essas instruções são escritas em linguagens de programação, que variam em complexidade e finalidade.



Os fundamentos valem para todas

Os conceitos básicos que vamos aprender são os pilares de qualquer linguagem e de qualquer aplicação.

A importância da programação



Está em tudo

Presente em praticamente todas as áreas da nossa vida: do celular aos servidores, até o micro-ondas.



Muda vidas

Traz liberdade criativa para tornar qualquer ideia realidade: um app, um protótipo, um negócio.



Forma de pensar

Vai muito além do código: é uma forma de pensar e resolver problemas.

Algoritmo: um passo a passo

Um algoritmo é uma sequência de instruções para resolver um problema. Por exemplo, ligar o projetor:



1. Conectar

Ligar o cabo do projetor no computador.

2. Energia

Conectar o projetor na tomada.

3. Ligar

Apertar o botão de ligar.

4. Entrada

Escolher a entrada certa (HDMI).

Se a ordem mudar ou um passo faltar, não funciona. Com o computador é igual.



Ambiente de Desenvolvimento

O que é um ambiente de desenvolvimento?

É o ambiente onde os desenvolvedores escrevem e testam o código de suas aplicações.



Na cozinha

Você tem os utensílios, ingredientes e equipamentos necessários para preparar uma refeição.



No desenvolvimento

Você tem as ferramentas essenciais para escrever, executar e testar código de maneira eficiente.

O que compõe um bom ambiente?



Editor de código

Onde você escreve o código, com realce de sintaxe, indentação e autocompletar.



Compilador ou interpretador

Transforma o seu código em instruções que o computador entende e executa.



Depurador

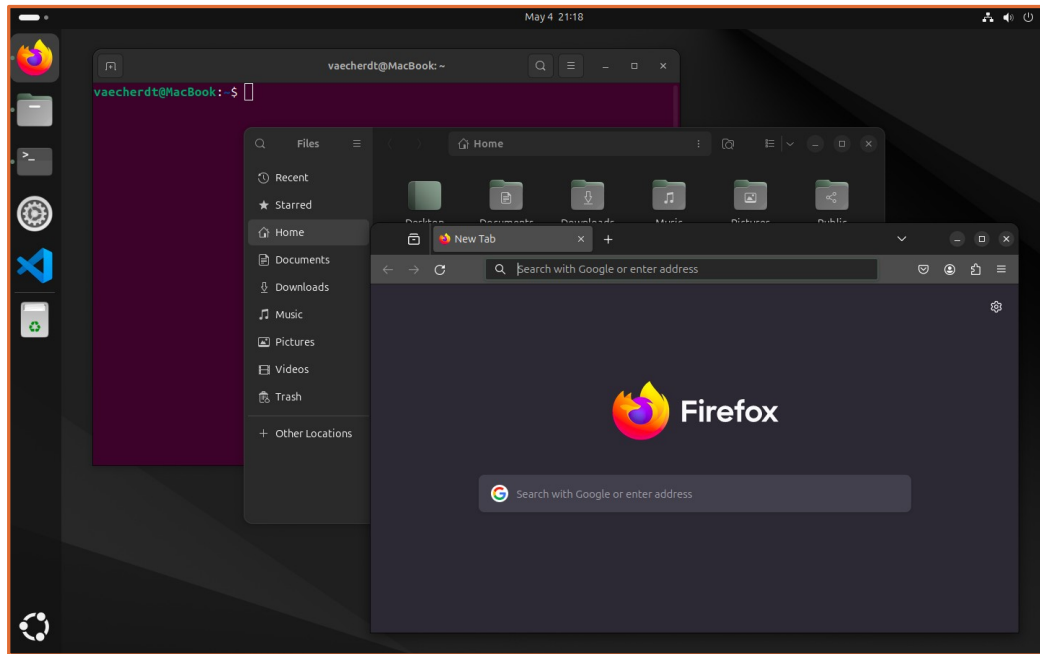
Ajuda a encontrar e corrigir erros no código, passo a passo.



Ambiente de execução

Onde o código roda e é testado, simulando o uso real.

Qual sistema operacional vamos usar?



- **Nos computadores do curso usamos o Linux, na distribuição Ubuntu.**
- **É gratuito, de código aberto e muito usado em servidores.**
- **Grande parte do backend roda em servidores Linux: aprender a usá-lo desde já é uma vantagem.**

Editores e IDEs mais usados



IntelliJ



Sublime Text



Vim



Atom



Emacs



Eclipse



Notepad++



VS Code

Por que o Visual Studio Code?



Leve e rápido

Abre rápido e consome poucos recursos do computador.



Ecossistema de extensões

Milhares de extensões para personalizar o editor do seu jeito.



Código aberto e comunidade ativa

Gratuito, atualizado todo mês e com muito conteúdo na internet.

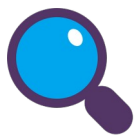


Suporte a JavaScript

Autocompletar, realce de sintaxe e depurador integrados.



Como instalar o VS Code



1. Pesquisar

Acesse o site oficial do VS Code.

2. Baixar

Baixe o arquivo **.deb**, o formato do Ubuntu.

3. Instalar

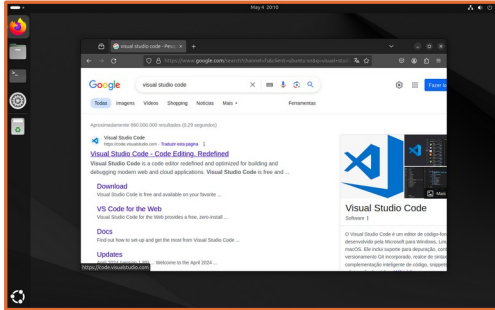
Instale pelo terminal, com o **apt**.

4. Abrir

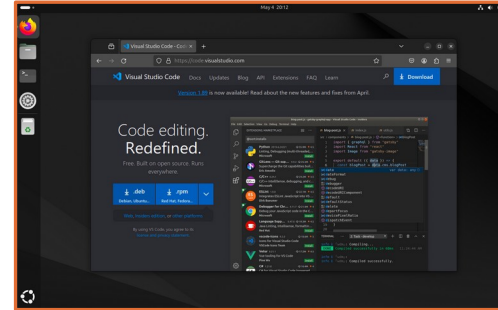
Digite **code** no terminal.

Site: code.visualstudio.com • **Instalação, na pasta do download:** `sudo apt install ./code_*.deb` • **code** . abre o VS Code na pasta atual.

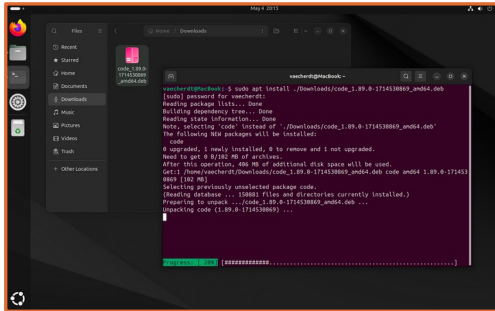
Instalação passo a passo



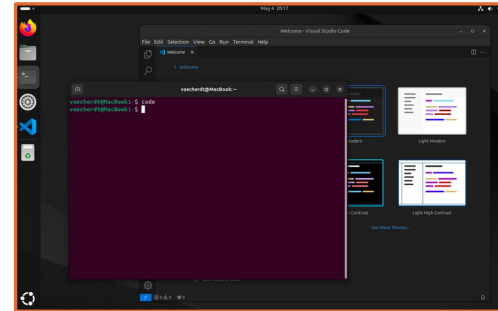
1. Pesquisar o site para download



2. Baixar o arquivo .deb para Ubuntu

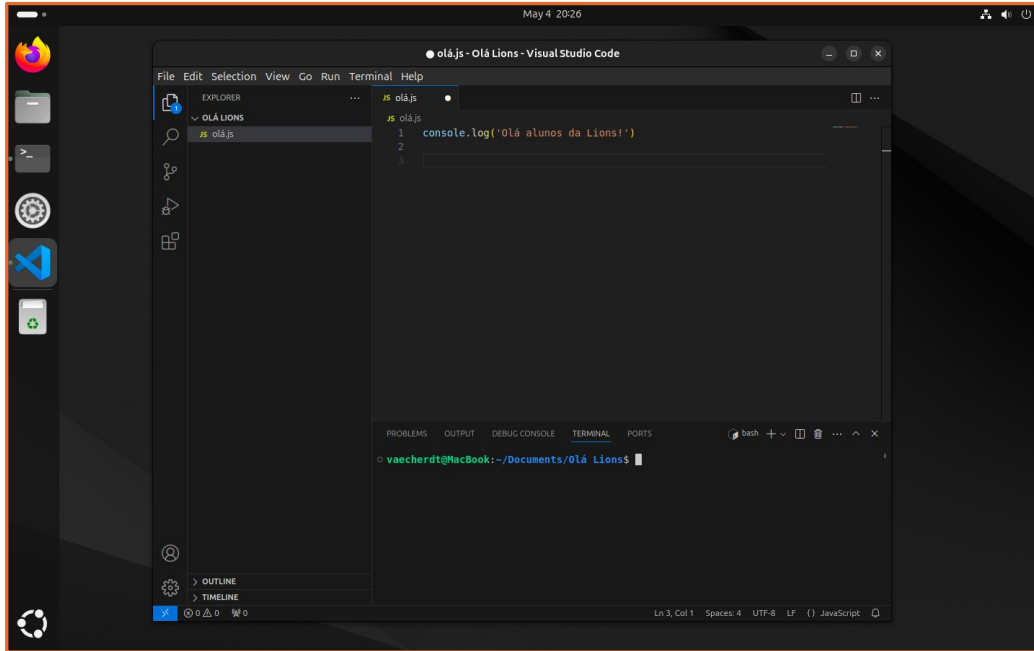


3. Instalar com sudo apt install



4. Abrir com o comando code

Conhecendo o VS Code



- **Explorer (à esquerda):** navega pelas pastas e arquivos do projeto.
- **Editor (ao centro):** o arquivo aberto, com uma aba para cada arquivo.
- **Terminal integrado (embaixo):** o terminal dentro do próprio VS Code.

Escrevemos o código no VS Code. Mas quem vai interpretar e executar esse código?

Node.js



JavaScript fora do navegador

Permite executar código JavaScript no servidor, indo além do navegador.



Eficiente

Lida com um grande número de conexões simultâneas sem sobrecarregar o sistema.



Pacotes prontos

Milhares de pacotes disponíveis através do npm.



Como o JavaScript se relaciona com o Node.js?



Linguagem de programação

Desenvolvida para ser usada no navegador.

Muito usada no front-end.



Ambiente de execução

Permite executar JavaScript fora do navegador.

Muito usado no back-end.

Comparar os dois é como comparar maçãs e laranjas: um é a língua, o outro é onde ela é falada.

npm: o gerenciador de pacotes



Gerenciador de pacotes do Node.js

Vem instalado junto com o Node.js.



Instala e organiza pacotes

Bibliotecas e frameworks prontos, instalados com um comando.



npm Registry

Um repositório online onde qualquer pessoa pode publicar e usar pacotes.





**Como o desenvolvimento web
funciona?**

Front-end x Back-end



Front-end

- Interação direta com o usuário.
- Botões, formulários, layouts.
- HTML, CSS e JavaScript.
- Executado no navegador.



Back-end

- Os "bastidores" da aplicação.
- Regras de negócio e processamento de dados.
- JavaScript (Node.js), Python, Java, Ruby...
- Executado no servidor.

Os dois trabalham juntos para criar uma experiência funcional e agradável para o usuário.

Pense em um carro



Front-end

O que o usuário vê e toca: o painel, as luzes, os botões.

Back-end

O que faz tudo funcionar: o motor e a transmissão.

Instalando o Node.js e o npm

A instalação é feita pelo terminal. Usamos o repositório do NodeSource, que traz a versão LTS (estável) mais recente:

```
$ curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -  
$ sudo apt install -y nodejs
```

O Node.js do NodeSource já vem com o npm. O `sudo apt install nodejs` direto do Ubuntu costuma instalar uma versão antiga.



Cuidado: a senha do sudo não aparece

Ao digitar a senha do **sudo**, **nada aparece** na tela. O teclado não travou!

- **sudo** executa o comando como administrador, por isso pede a sua senha.
- Digite a senha normalmente e aperte Enter, mesmo sem ver nada.
- Em algumas versões do Ubuntu aparecem asteriscos; em outras, nem isso.

```
$ sudo apt install -y nodejs  
[sudo] password for aluno:
```

Conferindo a instalação

```
$ node -v  
v24.x.x
```

```
$ npm -v  
11.x.x
```

node -v → mostra a versão do Node.js instalada.

npm -v → mostra a versão do npm.

versão → o número depende da LTS do momento: 24 até outubro de 2026, depois 26. Anote a sua: alguns comportamentos mudam entre versões.



Hora da Prática

Atividade Prática 1

Ambiente

1. Se estiver usando máquina virtual, importe a VM no VirtualBox.
2. Instale o VS Code e abra-o pelo terminal com o comando `code`.
3. Instale o Node.js (o npm vem junto).
4. Confira as versões com `node -v` e `npm -v`.

Revisão

? O que é programação?

Criar instruções para o computador executar tarefas.

? Qual é o nosso ambiente?

O VS Code, rodando no Ubuntu.

? Quem executa o código?

O Node.js, nosso ambiente de execução.

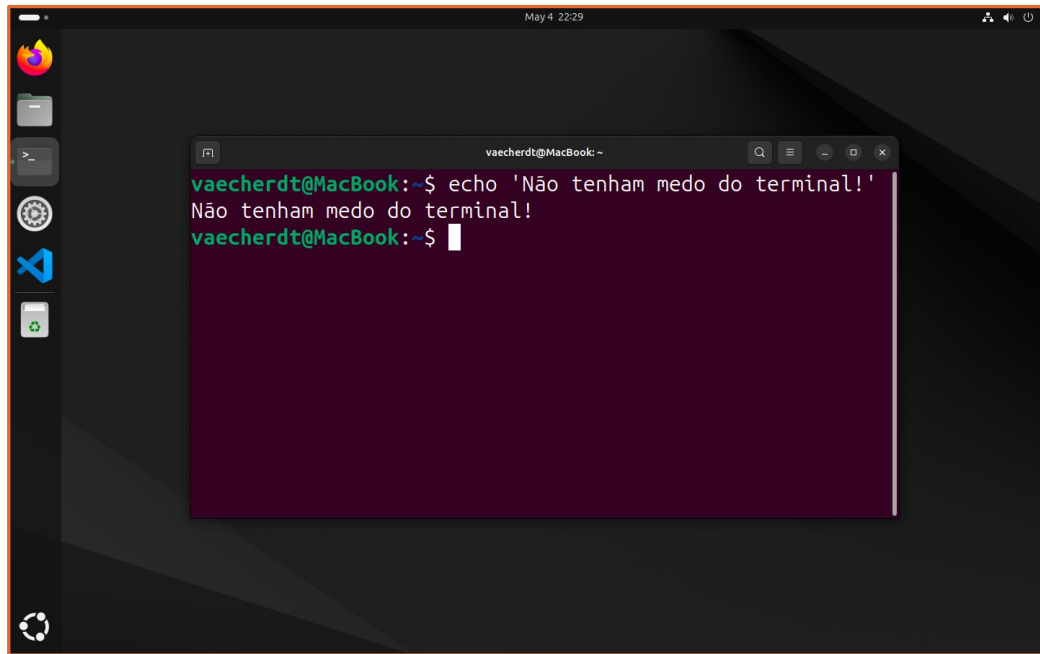
? O que é o npm?

O gerenciador de pacotes do Node.js.



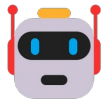
Introdução ao Terminal

O que é o terminal?



- **Uma interface de texto que permite interagir com o sistema.**
- **Nele executamos comandos e automatizamos tarefas.**
- **Curiosidade: usamos o terminal para configurar todos os PCs desta sala. Ele nos economizou muitas horas!**

O terminal no desenvolvimento web



Automatização de tarefas

Scripts que fazem sozinho tarefas repetitivas.



Gestão de projetos

Criar, navegar e manipular pastas e arquivos.



Controle de versão

O Git, que veremos no Módulo 4, é usado pelo terminal.



Instalação de dependências

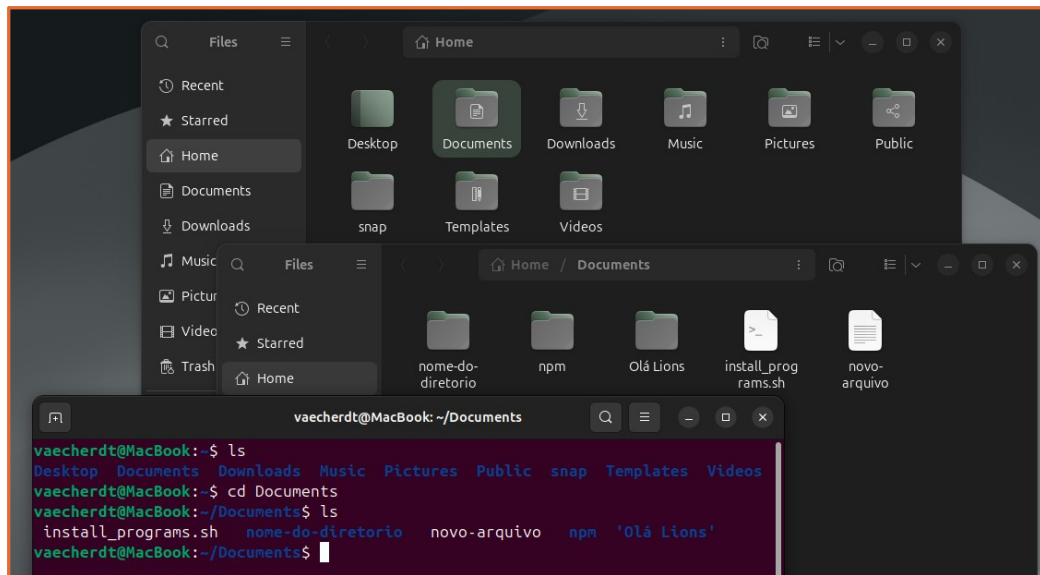
Instalar e gerenciar os pacotes do projeto com o npm.



Servidores locais e remotos

Iniciar servidores, rodar testes, ver logs e acessar servidores remotos.

Como ele funciona?



- O terminal está sempre dentro de uma pasta, assim como a janela de arquivos.
- O que você vê na janela, o terminal mostra em texto: aqui, **ls** lista o mesmo conteúdo da pasta Documents.
- Antes de rodar um comando, confira onde você está.

Anatomia de um comando

~/Desktop\$ **cp** pasta/arquivo.txt outra-pasta

Onde você está

Comando

Argumento 1

Argumento 2

```
$ mkdir nova-pasta  
$ cd pasta  
$ cp pasta/arquivo.txt outra-pasta  
$ echo "Texto aqui!" > arquivo.txt
```

Os argumentos são separados por espaço.



Atenção: maiúsculas e extensões importam

No Linux, `Arquivo.txt`, `arquivo.txt` e `Arquivo` são três arquivos diferentes.

- O Linux diferencia maiúsculas de minúsculas: é case-sensitive.
- A extensão (`.txt`, `.js`) faz parte do nome do arquivo.
- Dica: digite o começo do nome e aperte Tab: o terminal completa sozinho e evita erros de digitação.

Atalhos do terminal

Ctrl + Shift + C

Copiar o texto selecionado.

Ctrl + Shift + V

Colar no terminal.

Ctrl + C

Interrompe o comando que está rodando. Não é copiar!

Tab

Completa nomes de pastas e arquivos.

Seta para cima

Traz de volta os comandos anteriores.

clear

Limpa a tela (ou Ctrl + L).



Cuidado: espaços e acentos nos nomes

`mkdir Minha Pasta` cria **duas** pastas: "Minha" e "Pasta".

- Quando o nome tiver espaço, use aspas: `cd "Área de Trabalho"`.
- Os nomes mudam com o idioma do sistema: `Documentos` ou `Documents`, `Área de Trabalho` ou `Desktop`.
- Nos seus projetos, prefira nomes sem espaço e sem acento: `minha-pasta`.

```
$ mkdir Minha Pasta      # cria "Minha" e "Pasta"
$ mkdir "Minha Pasta"    # cria uma pasta só
$ mkdir minha-pasta      # melhor ainda
```

Comandos básicos

cd

Muda de pasta.

mkdir

Cria uma pasta.

ls

Lista o conteúdo da pasta.

pwd

Mostra onde você está.

touch

Cria um arquivo vazio.

cp

Copia um arquivo.

echo

Escreve uma mensagem.

rm

Remove um arquivo (pasta com **-r**).

Caminho absoluto x relativo

Absoluto: começa na raiz (/)

```
$ cd /home/aluno/Documentos
```

Relativo: a partir de onde você está

```
$ cd Documentos
```

O caminho absoluto funciona de qualquer lugar. O relativo depende da pasta atual. O ~ é um atalho para a sua pasta pessoal (/home/aluno).

Como voltar?

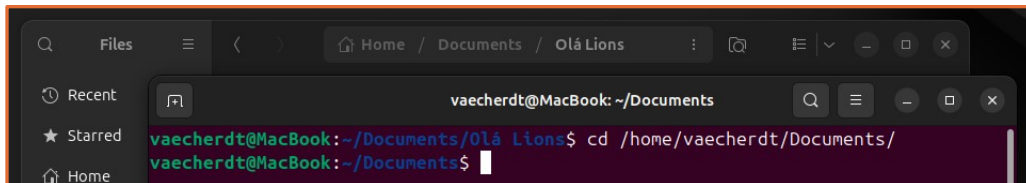
```
$ cd ..           # volta uma pasta
$ cd ../Desktop   # volta uma pasta e entra em Desktop
$ cd ~            # vai direto para a pasta pessoal
```

Os dois pontos .. significam "a pasta de cima".

cd: navegando entre pastas

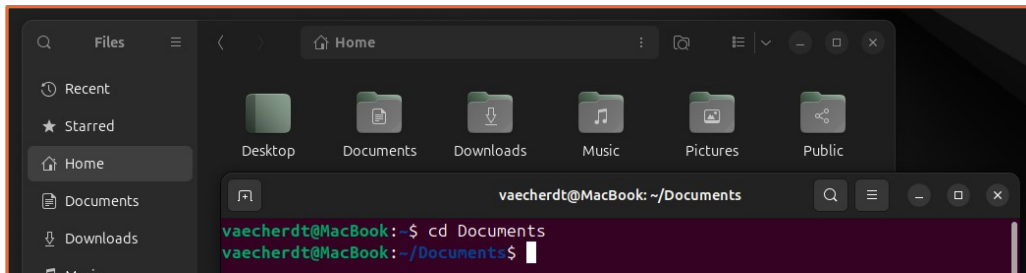
cd /caminho

Caminho absoluto: informamos o caminho completo até a pasta.



cd pasta

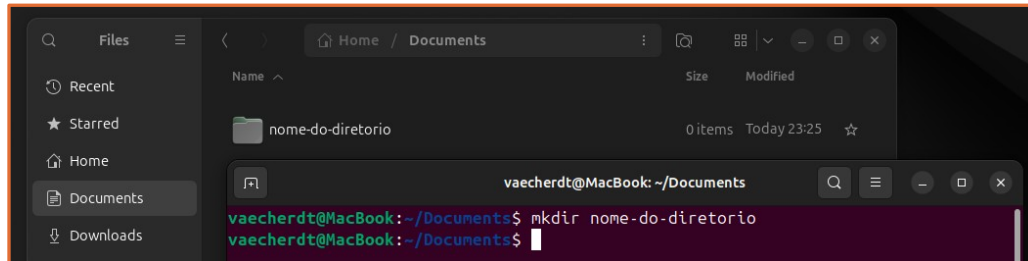
Caminho relativo: para entrar numa pasta que está dentro da atual.



mkdir e ls

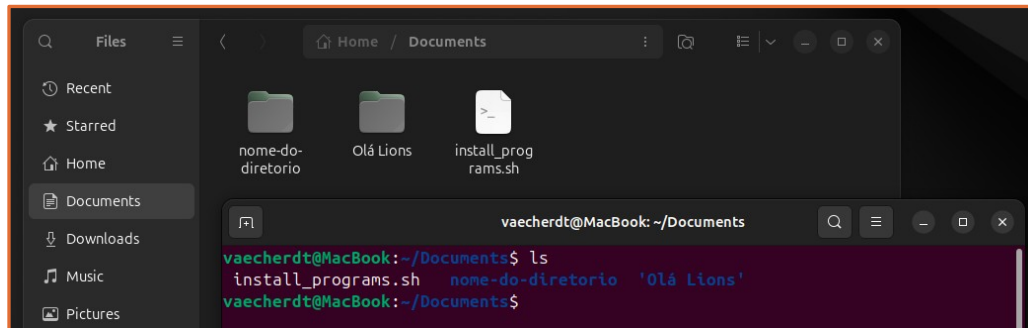
mkdir nome

Cria uma pasta dentro da pasta atual.



ls

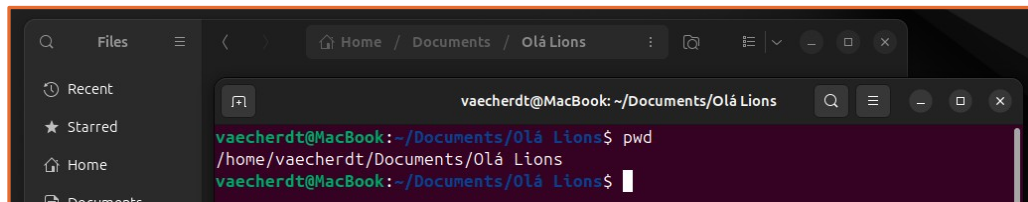
Lista o conteúdo da pasta aberta.



pwd e touch

pwd

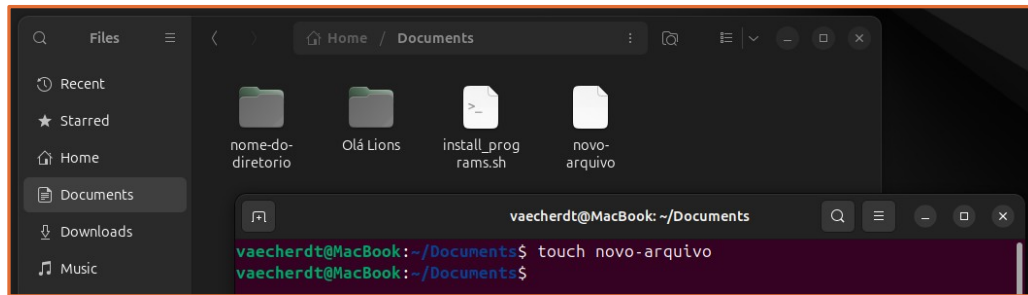
Mostra o caminho completo da pasta aberta.



```
vaecherdt@MacBook: ~/Documents/Olá Lions
vaecherdt@MacBook:~/Documents/Olá Lions$ pwd
/home/vaecherdt/Documents/Olá Lions
vaecherdt@MacBook:~/Documents/Olá Lions$
```

touch arquivo

Cria um arquivo vazio, com qualquer extensão (ou atualiza a data de um que já existe).

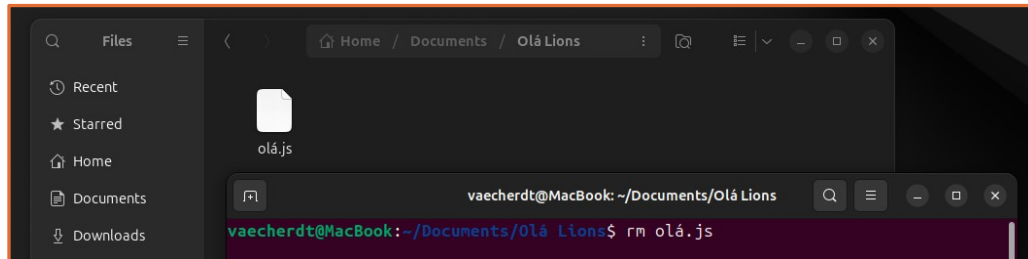


```
vaecherdt@MacBook: ~/Documents
vaecherdt@MacBook:~/Documents$ touch novo-arquivo
vaecherdt@MacBook:~/Documents$
```

rm: removendo

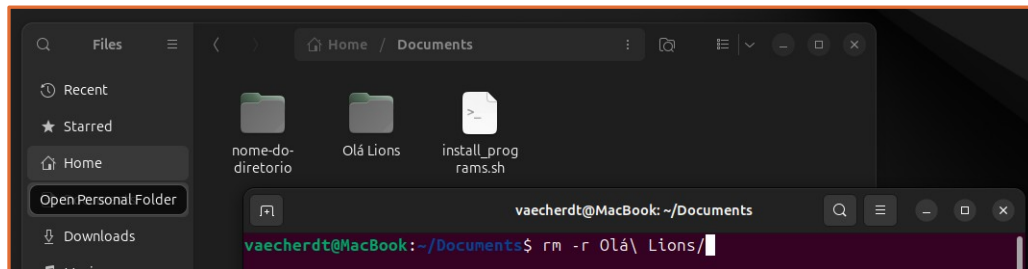
rm arquivo

Remove um arquivo. Cuidado: não vai para a lixeira. Apagou, sumiu.



rm -r pasta

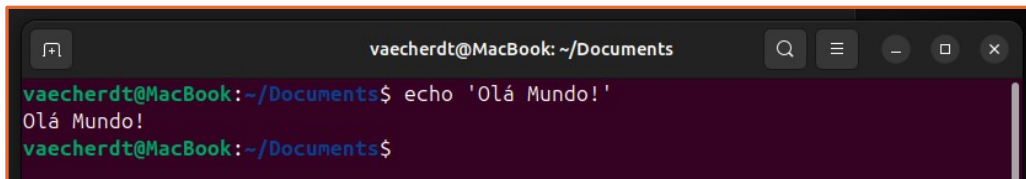
Remove a pasta e tudo que está dentro dela.



echo: escrevendo mensagens

echo "Olá"

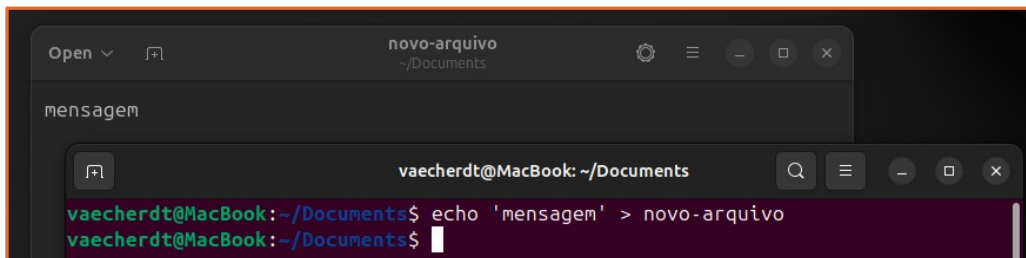
Imprime uma mensagem no terminal.

A screenshot of a macOS terminal window. The title bar shows 'vaecherdt@MacBook: ~/Documents'. The prompt is 'vaecherdt@MacBook:~/Documents\$'. The command 'echo 'Olá Mundo!'' has been entered and executed. The output 'Olá Mundo!' is displayed on the next line. The prompt is now 'vaecherdt@MacBook:~/Documents\$' again.

```
vaecherdt@MacBook:~/Documents$ echo 'Olá Mundo!'  
Olá Mundo!  
vaecherdt@MacBook:~/Documents$
```

echo "Oi" > a.txt

Com >, escreve a mensagem dentro do arquivo, substituindo o conteúdo. Com >>, adiciona no final.

A screenshot showing a macOS file editor window titled 'novo-arquivo' with the content 'mensagem'. Below it, a terminal window is open with the title 'vaecherdt@MacBook: ~/Documents'. The prompt is 'vaecherdt@MacBook:~/Documents\$'. The command 'echo 'mensagem' > novo-arquivo' has been entered and executed. The prompt is now 'vaecherdt@MacBook:~/Documents\$' again.

```
vaecherdt@MacBook:~/Documents$ echo 'mensagem' > novo-arquivo  
vaecherdt@MacBook:~/Documents$
```

Mais comandos úteis

```
$ cat BoasVindas.txt           # mostra o conteúdo do arquivo
Bem-vindos à Estação Espacial!
```

```
$ mv velho.txt novo.txt        # renomeia
$ mv novo.txt Laboratorio/      # move para outra pasta
$ cp -r Habitacao Backup       # copia uma pasta inteira
```

O cp sozinho não copia pastas: para isso, use cp -r. E o cat é ótimo para conferir se o echo funcionou.



Hora da Prática

Atividade Prática 2

Terminal

1. Crie duas pastas: original e copia (mkdir).
2. Entre na pasta original (cd).
3. Crie o arquivo index.txt (touch).
4. Coloque algum conteúdo no index.txt (echo).
5. Copie o arquivo para a pasta copia (cp).
6. Apague o arquivo da pasta original (rm).



Dica: relembRANDo: `cd Pasta, mkdir Nome, touch arquivo.txt, echo "Mensagem" > arquivo.txt, cp arquivo.txt ../copia, rm arquivo.txt.`

Desafio · Explorando o Espaço

Terminal

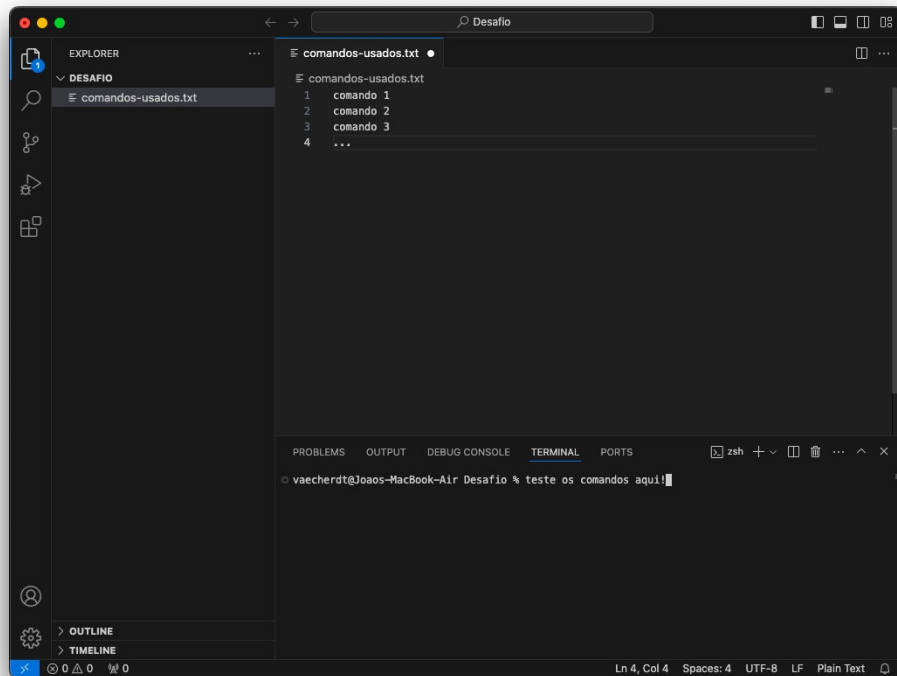
Represente uma estação espacial com pastas e arquivos, usando só o terminal:

1. **Crie a pasta `EstacaoEspacial` e, dentro dela, `Habitacao`, `Laboratorio` e `Hangar`.**
2. **Em `Habitacao`, crie `Quarto1`, `Quarto2` e `Quarto3`. Em `Laboratorio`, crie `Pesquisa1` e `Pesquisa2`.**
3. **Em `Hangar`, crie `Nave1`, `Nave2` e `Nave3`. Depois, remova o `Hangar` inteiro.**
4. **Na raiz, crie `BoasVindas.txt` com "Bem-vindos à Estação Espacial!" e copie para o `Laboratorio`.**
5. **Bônus: altere a cópia do Laboratório para "Bem-vindos à Estação Espacial! Este é o laboratório!".**



Dica: confira onde você está (`pwd`) antes de cada comando e use `cat` para conferir o conteúdo dos arquivos. Errou? Corrija com os próprios comandos.

Desafio: o que entregar



- Anote, em ordem, todos os comandos que você usou.
- Salve essa lista num arquivo **comandos-usados.txt**.
- Confira se as pastas e arquivos ficaram organizados como no enunciado.



Tema e Atalhos no VS Code

Por que um tema é importante?



Personalização visual

O editor com a sua cara.



Foco e produtividade

Menos cansaço, mais concentração.



Preferências individuais

Claro, escuro, alto contraste...



Identificação de código

Cada parte do código com uma cor.



Acessibilidade

Temas para daltonismo e baixa visão.



Estilo e motivação

Um ambiente agradável dá vontade de programar.

E se eu escolher um tema ruim?



Fadiga ocular

Cores fortes cansam a vista em poucas horas.



Legibilidade

Pouco contraste dificulta ler o código.



Sintaxe confusa

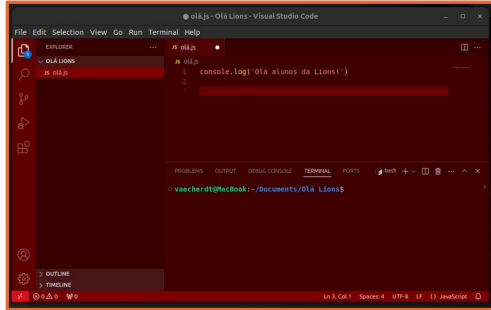
Fica difícil distinguir cada parte do código.



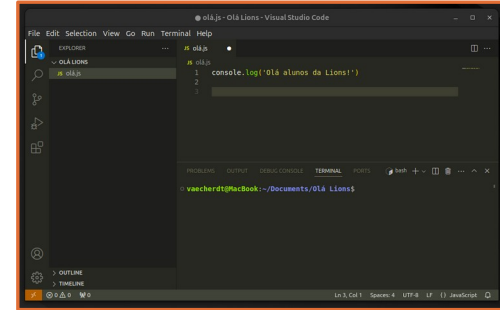
Distração

Cores demais tiram o foco do que importa.

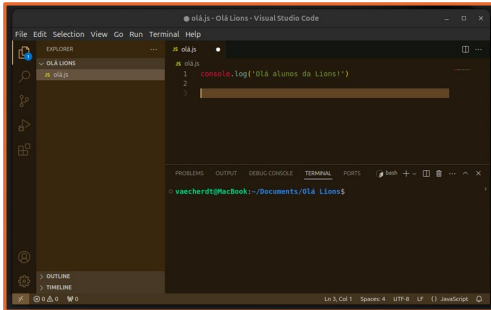
Exemplos de temas



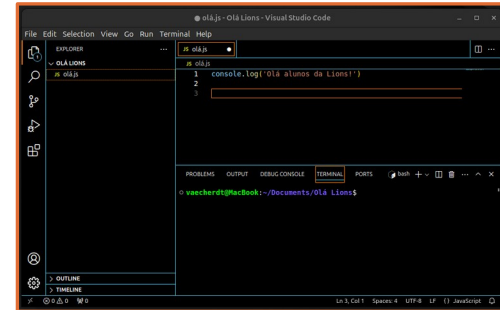
Vermelho intenso



Tons de cinza e verde

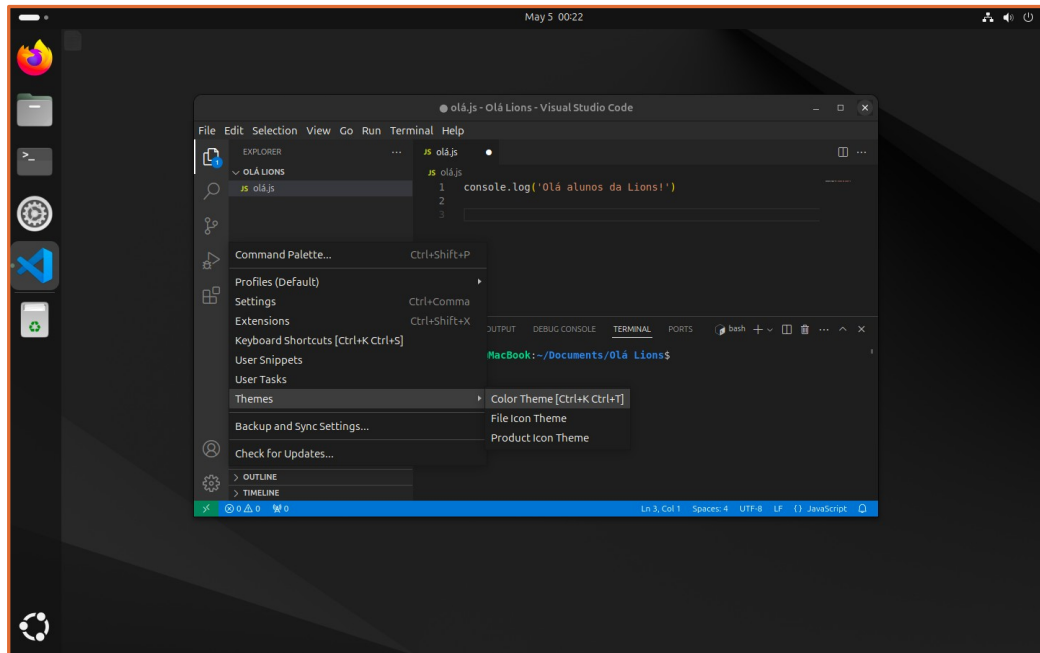


Tons de marrom



Alto contraste

Como escolher meu tema?



- Menu Arquivo > Preferências > Tema > Tema de Cores.
- Ou pelo atalho **Ctrl + K** e, em seguida, **Ctrl + T**.
- Use as setas para testar os temas antes de confirmar.

Atalhos úteis

Ctrl + S

Salvar o arquivo. Use sempre!

Ctrl + W

Fechar a aba aberta.

Ctrl + B

Esconder ou mostrar o painel lateral.

Ctrl + `

Esconder ou mostrar o terminal.

Ctrl + Shift + `

Abrir um novo terminal.

Ctrl + /

Comentar ou descomentar a linha.

A crase (`) fica na tecla ao lado do 1, no teclado brasileiro.



Cuidado: a bolinha na aba

Uma **bolinha** no lugar do X da aba indica que o arquivo **não** foi salvo.

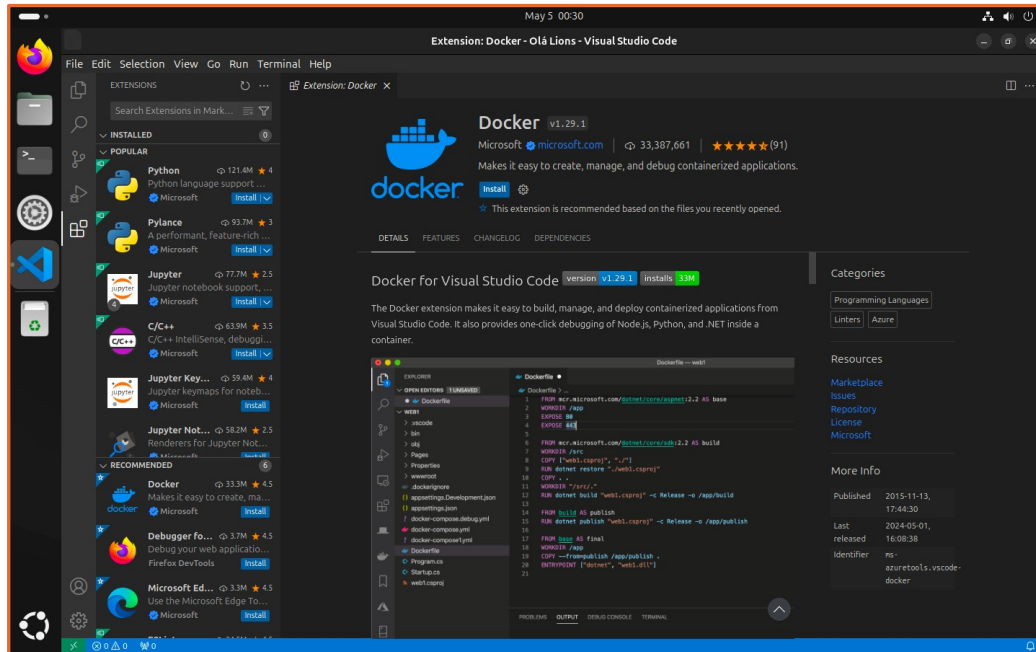
- O **node** executa o que está salvo no disco: sem salvar, você roda a versão antiga do código.
- Salve com **Ctrl + S** ou ative o salvamento automático em **Arquivo > Salvamento Automático**.



Extensões Úteis

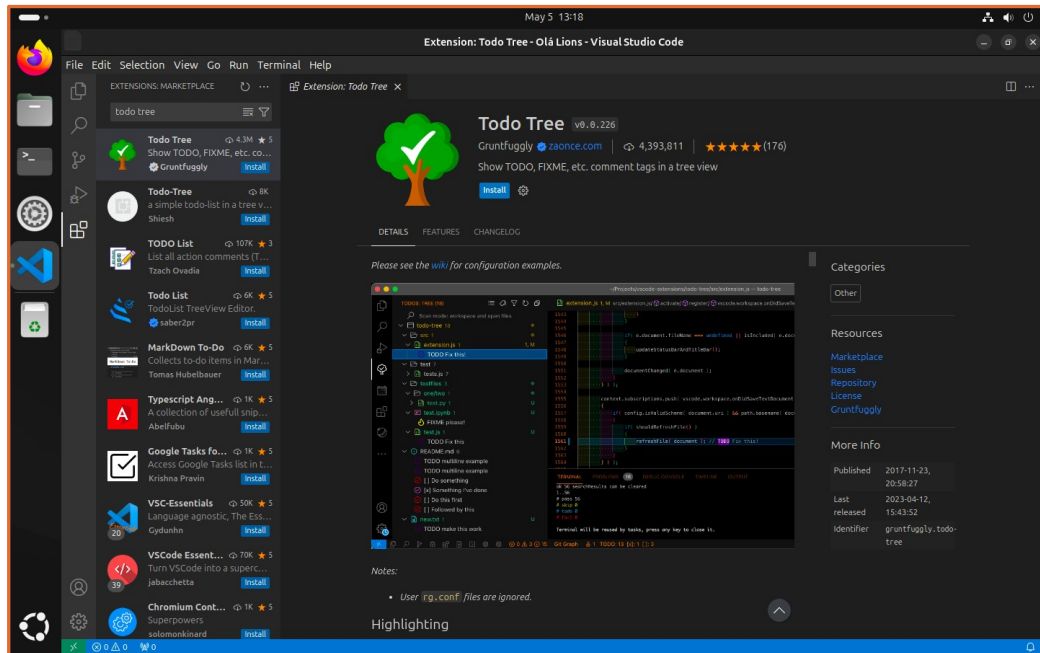
Para desenvolvimento web e mobile

O que são extensões?



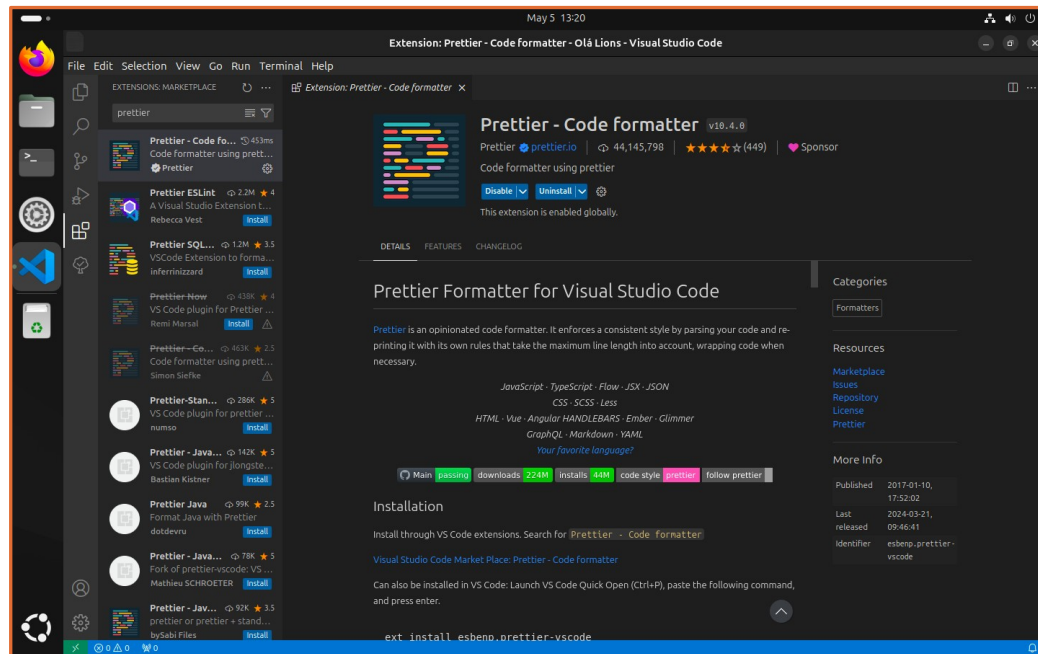
- Complementos que adicionam recursos ao VS Code.
- Ficam na aba Extensões, na barra lateral (**Ctrl + Shift + X**).
- Instale só o que for usar: extensão demais deixa o editor lento.

Todo Tree



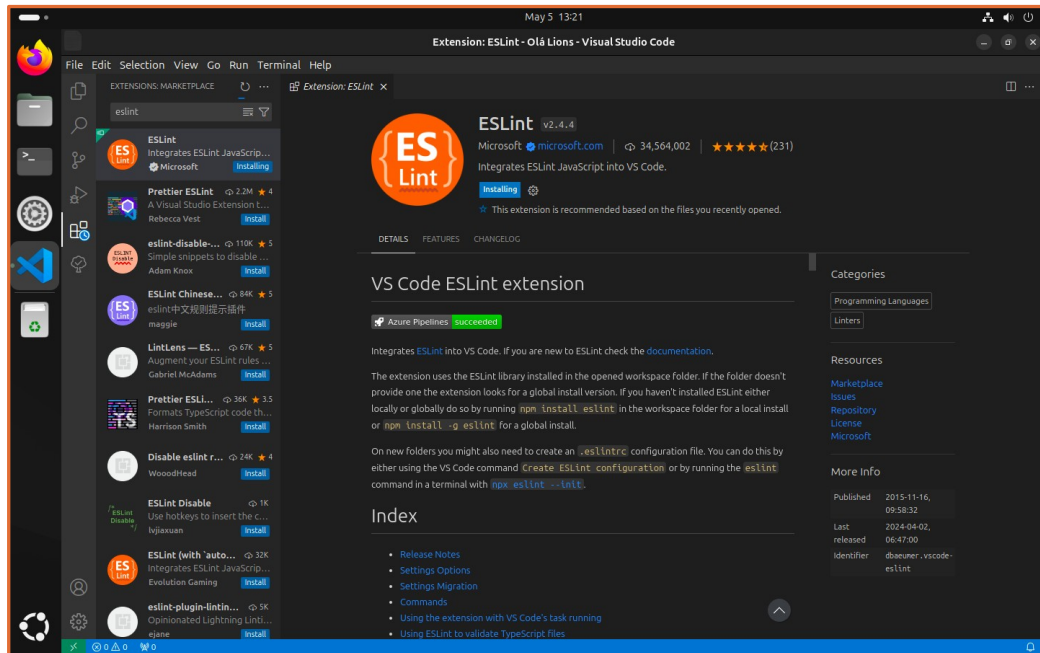
- Organiza as tarefas pendentes marcadas com **TODO**: no código.
- Mostra todas elas numa lista, em uma aba própria.
- Exemplo: **// TODO: validar a entrada do usuário**

Prettier



- Formata o código automaticamente, mantendo um estilo consistente.
- Indentação, aspas e espaços sempre do mesmo jeito, em todo o projeto.
- Dica: ative Format On Save nas configurações para formatar ao salvar.

ESLint



- Analisa o código e aponta padrões problemáticos e possíveis erros.
- Ajuda a manter o código consistente, principalmente em equipe.
- Os avisos aparecem sublinhados direto no editor.



Hora da Prática

Atividade Prática 3

VS Code

1. Abra o VS Code.
2. Instale as extensões Todo Tree, Prettier e ESLint.
3. Escolha e instale um tema.



Dica: procure temas no próprio marketplace de extensões, pesquisando por "theme".



Comandos Básicos do npm

Antes de mais nada...

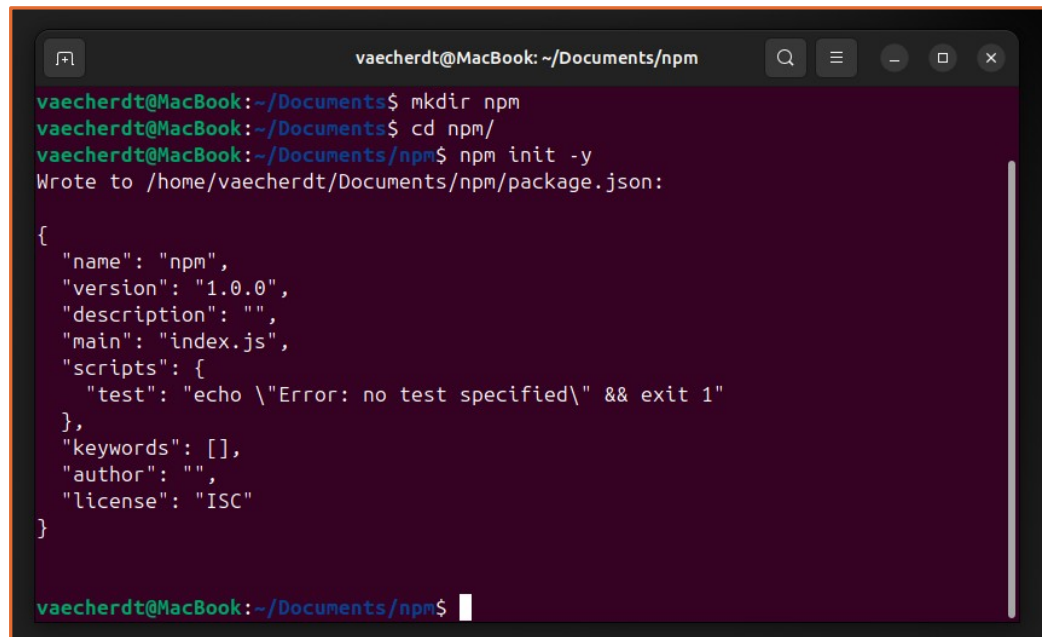
```
Documentos/  
├── Módulo1/  
├── Módulo2/  
└── Módulo3/
```

Documentos → todos os trabalhos e atividades ficam dentro desta pasta.

MóduloN → uma pasta para cada módulo do curso.

por quê? → fica fácil achar as atividades e ninguém mistura arquivos no computador compartilhado.

Como iniciar um projeto Node.js?



```
vaecherdt@MacBook: ~/Documents/npm
vaecherdt@MacBook:~/Documents$ mkdir npm
vaecherdt@MacBook:~/Documents$ cd npm/
vaecherdt@MacBook:~/Documents/npm$ npm init -y
Wrote to /home/vaecherdt/Documents/npm/package.json:

{
  "name": "npm",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

vaecherdt@MacBook:~/Documents/npm$
```

- Crie uma pasta para o projeto.
- Abra essa pasta no terminal.
- Execute **npm init -y**: ele cria o arquivo de configuração do projeto, o package.json.



Cuidado: confira a pasta antes do npm init

Um **package.json** criado na pasta pessoal vale para **todos** os projetos dentro dela.

- O **npm install** pode acabar instalando os pacotes lá na pasta pessoal, e não no seu projeto.
- No Módulo 2, o aviso do **import** vai apontar justamente para esse arquivo perdido.
- Rode **pwd** antes. Criou no lugar errado? Apague com **rm package.json**.

```
$ pwd  
/home/aluno/Documentos/Modulo1/meu-projeto  
$ npm init -y
```

O que é o package.json?

```
{
  "name": "npm",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

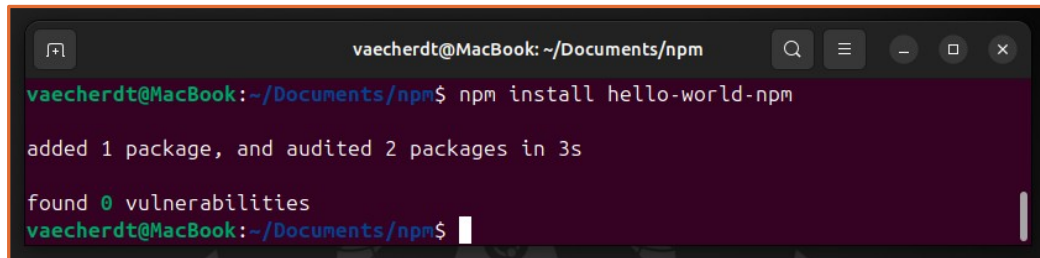
name / version → o nome e a versão do projeto.

main → o arquivo principal.

scripts → atalhos de comandos (veremos mais adiante).

dependencies → aparece quando você instala o primeiro pacote.

Instalando um pacote

A screenshot of a macOS terminal window. The title bar shows 'vaecherdt@MacBook: ~/Documents/npm'. The prompt is 'vaecherdt@MacBook:~/Documents/npm\$'. The command entered is 'npm install hello-world-npm'. The output shows 'added 1 package, and audited 2 packages in 3s' and 'found 0 vulnerabilities'. The prompt is now 'vaecherdt@MacBook:~/Documents/npm\$' with a cursor.

```
vaecherdt@MacBook: ~/Documents/npm
vaecherdt@MacBook:~/Documents/npm$ npm install hello-world-npm
added 1 package, and audited 2 packages in 3s
found 0 vulnerabilities
vaecherdt@MacBook:~/Documents/npm$
```

- Para instalar um pacote, usamos **npm install nome-do-pacote**.
- Aqui instalamos o **hello-world-npm**, um pacote de exemplo.
- O npm baixa o pacote e anota no package.json.



Cuidado: a pasta não pode ter o nome do pacote

Numa pasta chamada **prompt-sync**, o **npm install prompt-sync** **falha**.

- O **npm init -y** usa o nome da pasta como nome do projeto, e o npm se recusa a instalar um pacote com o mesmo nome do projeto.
- Dê à pasta um nome que descreva o seu projeto: **calculadora**, **atividade-1**.

```
$ npm install prompt-sync
npm error code ENOSELF
npm error Refusing to install package with name "prompt-sync" under
npm error a package also called "prompt-sync"
```

O que o npm install cria?

```
$ npm install prompt-sync  
added 3 packages in 1s
```

```
$ ls  
node_modules  package.json  package-lock.json
```

node_modules/ → pasta com os pacotes baixados. Não precisa mexer nela.

package-lock.json → guarda as versões exatas que foram instaladas.

dependencies → o pacote é anotado no package.json.

apagou? → o **npm install** baixa tudo de novo, lendo o package.json.

O campo "type"

```
{  
  "name": "meu-projeto",  
  "version": "1.0.0",  
  "main": "index.js",  
  "type": "commonjs",  
  "dependencies": {  
    "prompt-sync": "^4.2.0"  
  }  
}
```

"type" → diz ao Node.js como ler os arquivos **.js** do projeto.

"commonjs" → usa **require**. É o padrão quando o campo não existe.

"module" → usa **import**.

Módulo 2 → vamos usar isso quando formos importar pacotes.

Desinstalando um pacote

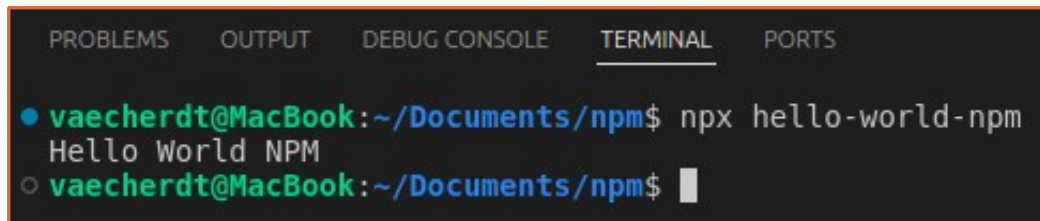
```
$ npm uninstall hello-world-npm
```

uninstall → remove o pacote e as dependências dele da pasta **node_modules**.

package.json → a entrada do pacote sai de **dependencies**.

cuidado → se o seu código usa esse pacote, ele vai parar de funcionar.

Comando npx



A screenshot of a VS Code terminal window. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal text shows a user named 'vaeherdt@MacBook' in the directory '~/Documents/npm' running the command 'npx hello-world-npm'. The output is 'Hello World NPM'. The prompt is followed by a cursor.

```
• vaeherdt@MacBook:~/Documents/npm$ npx hello-world-npm
Hello World NPM
○ vaeherdt@MacBook:~/Documents/npm$
```

- Executa um pacote sem precisar instalá-lo no projeto.
- O npm baixa o pacote, roda e descarta.
- Muito usado para ferramentas que você só roda de vez em quando.

Testando rápido: o modo interativo do Node

```
$ node  
> 2 + 3  
5  
> "Lions" + " Dev"  
'Lions Dev'  
> .exit
```

node → sem nome de arquivo, abre o **REPL**: você digita uma linha e vê o resultado na hora.

para quê → testar uma conta, um método ou uma ideia sem criar arquivo.

sair → **.exit** ou **Ctrl + C** duas vezes.

Aprendendo a aprender



Leia o erro

A mensagem diz o arquivo, a linha e o motivo. Comece por ela.



Pesquise a mensagem exata

Copie o erro e pesquise: alguém já passou por isso.



Documentação

developer.mozilla.org para JavaScript e nodejs.org/docs para o Node.



IA como tutor

Peça para explicar, não para fazer: você precisa entender o código que entrega.

No terminal, quase todo comando tem ajuda: `ls --help`, `npm help install`.

Resumo do módulo



Apresentações

A turma, a metodologia e a ementa.



Programação e ambiente

Algoritmos, editor, interpretador e depurador.



VS Code, Node.js e npm

Instalação e primeiros passos.



Terminal

cd, mkdir, ls, pwd, touch, cp, echo e rm.



Tema, atalhos e extensões

O VS Code do seu jeito.



npm na prática

init, install, uninstall, npx e package.json.

Para finalizar o Módulo 1...

Atividade Avaliativa

Módulo 1

1. Abra o terminal e navegue até a pasta Documentos.
2. Crie a pasta avaliacao e abra o VS Code dentro dela com `code ..`
3. Abra o terminal do VS Code e inicie um projeto com `npm init -y`.
4. Crie o arquivo `seu-nome.js` com o `touch`.
5. Com o `echo`, coloque dentro do arquivo a linha `console.log('eu consegui')`.
6. Execute o arquivo pelo terminal: `node seu-nome.js`.
7. Tire um print mostrando o terminal do VS Code, o arquivo e a estrutura de pastas.



Dica: use aspas duplas por fora e simples por dentro: `echo "console.log('eu consegui')" > seu-nome.js`.

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo01](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo01)



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

Dúvidas? Nos vemos no Módulo 2.